```
arguments)>
>> ledger.reset
RSpec::Mocks::MockExpectationError: (Double "Ledger").reset(no args)
    expected: 0 times with any arguments
    received: 1 time
    « backtrace truncated »
```

Here, we see a failure because the mock object received a message it was specifically expecting not to receive. With rspec-mocks, you can spell out much more fine-grained expectations than just receiving or not receiving messages. Later, you'll see how.

Null Objects

The test doubles you've defined so far are *strict*: they require you to declare in advance what messages are allowed. Much of the time, this is what you want. But when your test double needs to receive several messages, having to spell each one out can make your tests brittle.

In these situations, you may want a test double that's a little more forgiving. That's where null objects come in. You can convert any test double to a null object by calling `as_null_object` on it:

```
>> yoshi = double('Yoshi').as_null_object
=> #<Double "Yoshi">
>> yoshi.eat(:apple)
=> #<Double "Yoshi">
```

This type of null object is known as a *black hole*; it responds to any message sent to it, and always returns itself. This means that you can chain one method call after another, for as many calls as you care to:

```
>> yoshi.eat(:apple).then_shoot(:shell).then_stomp
=> #<Double "Yoshi">
```

Null objects are the placebos of the testing world. They're benign objects that do nothing, can stand in for anything, and can satisfy any interface.

This flexibility is useful for testing objects that have multiple collaborators. If you have a `ChatBot` class that interacts with a `room` and a `user`, you may want to test these collaborations separately. While you're focusing on the user-related specs, you can use a null object for the `room`.

Spies

One downside of traditional mocks is that they disrupt the normal Arrange/Act/Assert sequence you're used to in your tests. To see what we mean, type in the following definition of a `Game` class: